

Getting Started in Open Source Security Research

...

Kate Kligman

About this Workshop

- Learn how to get started making security contributions to Open Source software
- Learn strategies and techniques for discovering and reporting Open Source vulnerabilities
- Static analysis vulnerability detection workshop

Open Source Security Research Benefits

- Help others on a larger scale.
- Few barriers to participation.
- Some employers allow a % of open-source contribution time.
- Public record of work to showcase.

Open Source Security Research Contribution Steps

1. Discover an issue, document the steps, create a proof of concept.
2. Privately report the issue to the developer or their security contact.
3. After the issue is fixed, write up and publish it.

Open Source Project Selection

GitHub



SOURCEFORGE



Bitbucket



GitLab



Codeberg



Canonical

Launchpad

Step 1: Discover an issue

OWASP Top 10: A03:2021 – Injection

- User-supplied data is not validated, filtered, or sanitized by the application.
- Dynamic queries or non-parameterized calls without context-aware escaping are used directly in the interpreter.
- Hostile data is used within object-relational mapping (ORM) search parameters to extract additional, sensitive records.
- Hostile data is directly used or concatenated. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.

<https://owasp.org/Top10/>

SQL Best Practice: Prepared Queries

```
$email = validate_email($_GET['email']);
```

```
$safe_query = $db->prepare("SELECT * FROM users WHERE email = %s",  
    $email  
);
```

```
$users = $db->get_results($safe_query);
```

SQL Injection Format: Textbook Example

Pseudocode:

```
database_query_function("SQL STRING " + USER INPUT);
```

PHP:

```
$db->query("SELECT * FROM TABLE WHERE ID = " . $_GET['id']);
```

Note: \$_GET is the superglobal array of string http GET parameters

Eg: `$_GET['id'] -> "5"` or `$_GET['id'] = " OR 1=1 -- "`

SQL Injection: Proof of Concept

Injection:

```
$db->query("SELECT * FROM TABLE WHERE ID = " . $_GET['id']);
```

Proof of Concept:

<http://www.home.arpa/?id=OR 1=1 -->

or

[http://www.home.arpa/?id=1 AND SLEEP\(5\) --](http://www.home.arpa/?id=1 AND SLEEP(5) --)

Step 2: Report the Issue

- Find the security contact or development contact.
- Security contact:
 - Follow all posted instructions to send a confidential message.
 - They may handle communication and publication for you.
- Developer contact:
 - Send a direct, confidential communication.
 - Secure a timeline for a fix and publication.
- Don't:
 - Don't post the issue publicly or to a public issue queue.
 - Don't publicly write or blog about the issue until it is fixed.

Issue Reporting

- Clearly and concisely describe the vulnerability.
- Describe the potential impact.
- List steps to reproduce and/or include the proof of concept.
- Ask for a timeframe for fixes and publication if one is not given.
- File for a CVE unless the security contact will be doing this for you.

CVE - Common Vulnerabilities and Exposures

Required CVE Record Information

CNA: Patchstack OÜ

Published: 2024-12-06 **Updated:** 2024-12-06

Title: WordPress WordPress Auction Plugin Plugin <= 3.7 - SQL Injection Vulnerability

Description

Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') vulnerability in Owen Cutajar & Hyder Jaffari WordPress Auction Plugin allows SQL Injection. This issue affects WordPress Auction Plugin: from n/a through 3.7.

CWE 1 Total

[Learn more](#)

- CWE-89: CWE-89 Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

CVSS 1 Total

[Learn more](#)

Score	Severity	Version	Vector String
9.3	CRITICAL	3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:L

Product Status

[Learn more](#)

Vendor

Owen Cutajar & Hyder Jaffari

Product

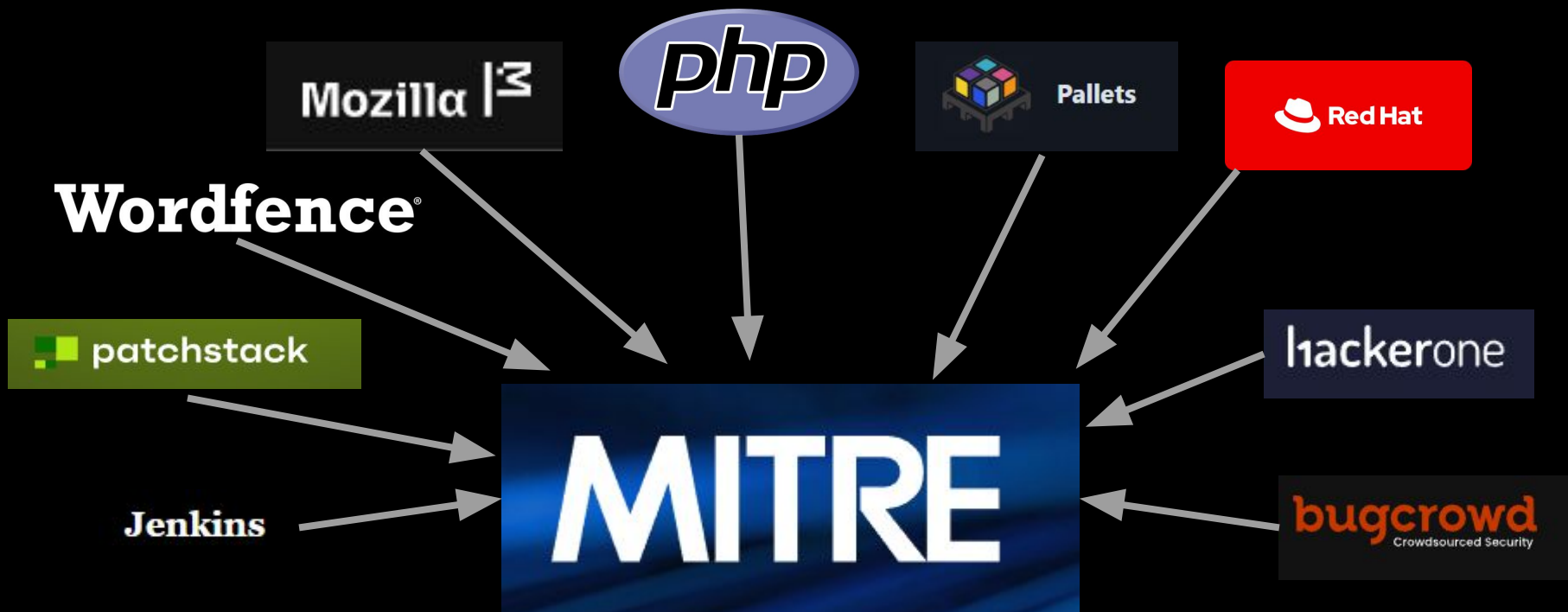
WordPress Auction Plugin

Versions 1 Total

Default Status: unaffected

Source: <https://www.cve.org/CVERecord?id=CVE-2024-51615>

CNA - CVE Numbering Authorities



Source: <https://www.cve.org/PartnerInformation/ListofPartners>

Step 3: Publish the Issue

- Developer publishes a fix along with release notes.
- CVE - Common Vulnerabilities and Exposures report.
- Blog post?



PHP Structure

```
$variable1 = $_GET['get_request_input'];  
$variable2 = $_POST['form_input'];  
$query = $variable1 . $variable2; // string concatenation with .
```

http://www.home.arpa?get_request_input=someinput

PHP Input Source: Superglobals

`$_GET` - array of items from a GET request

`$_POST` - array of items from a POST request

`$_COOKIE` - array of items from a cookie

`$_REQUEST` - merged array of `$_GET`, `$_POST`, `$_COOKIE`

`$_FILES` - array of items from file uploads

`$_SERVER['http_referer']` - HTTP Referer - user can modify this!

Note: The array values are strings!

PHP and WordPress Sanitizers

`$wpdb->prepare()`

`int()` or `(int)` typecast

`sanitize_email()`

`esc_attr()` and `esc_html()`

`strip_tags()`

`filter_var()`

Context matters for usage! Sanitizers are often not interchangeable!

PHP Output and Database Paths

```
<?php  
echo "Hello, World!";  
  
echo "A " . $variable . " and a {$variable} and " . $_GET['unsafe_input'];  
  
$wpdb->get_row("SELECT * FROM TABLE WHERE ID = " . $_GET['id']);  
  
$wpdb->query("UPDATE TABLE SET A=B WHERE ID = " . $unsure_if_evil);
```

Static Analysis Tools

- Scan code without running it.
- Use an AST (Abstract Syntax Tree) to navigate code.
- Rule-based analysis.
- Some can trace variables.
- Different tools can produce different results.

Static Analysis and Untrusted Variable Tracing

Input `$_GET, $_POST, $_REQUEST, $_COOKIE`



Sanitizer `$db->prepare(), intval(), filter_var(), sanitize_email()`



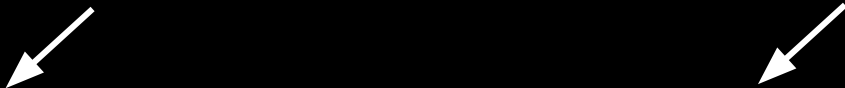
Sink `$db->query(), $db->get_row(), echo $output;`

SQL Injection: No Sanitizer

Pseudocode:

Sink

Source



```
database_query_function("SQL STRING " + USER INPUT);
```

The diagram shows two white arrows. One arrow points from the word 'Sink' down to the string literal '"SQL STRING "' in the pseudocode. The other arrow points from the word 'Source' down to the variable 'USER INPUT' in the pseudocode.

PHP:

```
$db->query("SELECT * FROM TABLE WHERE ID = " . $_GET['id']);
```

SQL Injections: Real World Example

```
$intAuctionID = $_GET["wpa_id"]; <-- Source
```

```
// get ready to reverse the last bid on the auction
```

```
$auction_table_name = $wpdb->prefix . "wpa_auctions";
```

```
// Step 1 - Update auction to set end timestamp to now
```

```
$sql = "UPDATE ".$auction_table_name." SET date_end =  
".current_time('mysql',"1")." WHERE id=". $intAuctionID;
```

```
$wpdb->query($sql); <-- Sink
```

CVE-2024-54292

```
if (isset($_GET['mobile']) && isset($_GET['code'])) {  
    $code = sanitize_text_field(wp_unslash($_GET['code']));  
    global $wpdb;  
    $table_name = $wpdb->prefix . "appsplate_checkout";  
    $item = $wpdb->get_row("SELECT * FROM $table_name  
WHERE code = '$code'");
```


Semgrep Static Analysis

- Corporate backed open-source. Free for public projects.
- Create search expressions based on abstract syntax trees.
- Rule-based static analysis.



Create a GitHub Codespace

- <https://github.com/katekligman/open-source-workshop>
- Click "Code" -> "Codespaces" tab
- Click "Create codespace on main"

